SARR-Core Engine

Plano profissional de implementação, arquitetura executável e prompts de desenvolvimento

---

Plataforma de Diagnóstico e Resiliência Defensiva
Purple Teaming • Auditoria autorizada • Telemetria • Mitigação controlada

Campo	Definição
Documento-base	Documento de Arquitetura, Casos de Uso e Fluxogramas.pdf (5 páginas)
Projeto	SARR-Core Engine
Versão deste plano	1.0 — 29/10/2026
Ambiente de desenvolvimento	Windows + Visual Studio Code + Git + GitHub Copilot
Primeira entrega recomendada	MVP local, visual, auditável e seguro; execução por .bat e modo portátil em pendrive
Classificação	Uso defensivo / auditoria autorizada

Objetivo: transformar a especificação de arquitetura em um plano de construção incremental, com requisitos verificáveis, estrutura de projeto, estratégia de instalação/execução e prompts prontos para uso no Copilot.

Importante: este documento é um plano de engenharia. Ele não afirma que o software já foi implementado, testado ou certificado. Cada fase deve produzir código, testes e evidências antes de avançar.

Preparado a partir do documento técnico fornecido pelo usuário.

1. Resumo executivo e decisões de implementação

A especificação-base descreve uma plataforma de segurança defensiva que combina validação contínua, auditorias controladas, telemetria, detecção de anomalias, resposta automática e trilhas de auditoria. Ela lista Wi-Fi 802.11, Bluetooth/BLE, rádio frequência/SDR, resiliência a carga L4/L7, Zigbee/Z-Wave, ARP/MitM, DNS sinkhole/spoof e motor preditivo de IA. Também exige validação de escopo/licença e um kill-switch antes e durante operações. filecite turn0file0 L15-L25

Para a primeira criação, recomenda-se NÃO implementar todos os vetores de baixo nível de uma só vez. A primeira versão deve ser um aplicativo local com interface gráfica, inventário de módulos, validação de escopo, telemetria simulada ou passiva, relatórios e controles de emergência. Módulos ativos só devem ser habilitados em fases posteriores, com autorização documentada, laboratório isolado e limites rígidos.

Decisão recomendada

Tema	Escolha inicial	Motivo
Linguagem	Python 3.12+ (versão exata a confirmar no ambiente)	Velocidade de prototipagem, testes e integração com bibliotecas de sistema.
Interface	PySide6 ou Tkinter; escolher uma e fixar no primeiro sprint	Aplicação desktop local, sem depender de navegador.
Persistência	SQLite	Banco local, simples de transportar e fácil de fazer backup.
Configuração	JSON validado por schema	Escopo e configurações legíveis e verificáveis.
Logs	JSON Lines + hash encadeado opcional	Facilita análise e detecção de alterações; não alegar imutabilidade absoluta em disco local.
Testes	pytest + testes de integração com simuladores	Verificação automatizada e reprodutível.
Distribuição	Modo de desenvolvimento, .bat e pacote portátil; instalador depois	Reduz complexidade inicial e facilita validar em Windows.

O documento-base especifica metas como latência HTTP de 1.500 ms para disparo do kill-switch e taxa de erros HTTP 5xx acima de 5% para interrupção. Também registra KPI de RTT inferior a 400 ms em pico de carga. Esses valores devem ser configuráveis por perfil de teste e tratados como requisitos iniciais a confirmar com o responsável pelo ambiente, não como garantias universais. filecite turn0file0 L101-L110 filecite turn0file0 L141-L155

Princípio de segurança: começar em modo somente leitura/simulação. Qualquer ação que altere firewall, switch, DNS, rádio, dispositivo ou serviço deve ficar desativada por padrão, exigir autorização explícita e oferecer simulação/dry-run.

2. Escopo funcional organizado por módulos

A lista abaixo preserva os vetores do documento-base e os organiza por dependência, risco e ordem de construção. A presença de um módulo no roadmap não significa que sua execução ativa esteja liberada por padrão.

Módulo	Funções previstas	Primeiro MVP	Fase posterior / restrição
Core Engine	Orquestração, estado de execução, eventos, políticas, configuração.	Sim: núcleo local e máquina de estados.	Obrigatório antes de qualquer integração ativa.
Escopo e licença	Validar scope.json, licença, alvos autorizados e validade.	Sim: bloqueio fail-closed.	Não permitir bypass manual pelo UI.
Wi-Fi 802.11	Inventário passivo de SSID/BSSID/canal/RSSI; comparação com allowlist; alerta de AP suspeito.	Dados importados ou leitura passiva compatível com o sistema.	Sem desautenticação, captura de credenciais ou testes fora de autorização.
Bluetooth/BLE	Descoberta e inventário de serviços/atributos GATT; avaliação de configuração.	Simulação e inventário consentido.	Sem interação invasiva com dispositivos não autorizados.
RF/SDR	Análise espectral, identificação de ruído e avaliação defensiva de replay.	Somente importação de medições ou dados de laboratório.	Transmitir/interferir em RF requer equipamento, autorização e ambiente legalmente adequado.
Resiliência L4/L7	Medição de RTT, erros, taxa de requisições e circuit breaker.	Somente teste local contra servidor de demonstração controlado.	Sem flood, esgotamento de conexões ou alvo público; limites e parada automática.
Zigbee/Z-Wave	Inventário autorizado de nós e configuração de rede mesh.	Dados simulados/importados.	Operações de re-keying apenas via integração aprovada.
ARP/MitM	Detectar indicadores de conflito ARP/MAC e sugerir controles.	Detecção passiva/telemetria simulada.	Não executar envenenamento ARP nem interceptar tráfego real.
DNS sinkhole	Identificar domínios de teste e simular redirecionamento em sandbox.	Simulador local e lista de domínios reservados para laboratório.	Não alterar DNS corporativo sem change request aprovado.
Motor preditivo IA	Anomalias em séries temporais e recomendação de limites.	Baseline estatístico explicável; modo recomendação.	Não aplicar bloqueios autônomos sem aprovação e rollback.
Mitigação	Propostas de regra, webhook, isolamento e reversão.	Dry-run, aprovação humana e registro de decisão.	Execução real requer RBAC, credenciais protegidas e rollback.
SIEM/Relatórios	Eventos, alertas, exportação JSON/CSV/PDF e trilha de auditoria.	Sim: eventos locais e relatório.	Conectores externos somente após especificar API e autenticação.

3. Arquitetura lógica recomendada

A aplicação deve separar interface, regras de negócio, conectores de sistema e persistência. O núcleo não deve executar comandos de sistema diretamente a partir de callbacks da interface. Todas as ações devem passar por serviços tipados, política de autorização, validação de escopo e registro de auditoria.

Camada	Responsabilidade	Contratos / componentes
UI Desktop	Painel inicial, estado do sistema, módulos, escopo, alertas, relatórios e configurações.	Dashboard, Scope Viewer, Run Console, Event Viewer, Reports.
Application Services	Coordena casos de uso e valida a sequência de execução.	RunService, ScopeService, ReportService, MitigationApprovalService.
Policy & Safety	Bloqueia operações sem escopo, fora dos limites ou sem autorização.	PolicyEngine, KillSwitch, RateLimiter, DryRunGuard.
Core Engine	Máquina de estados e barramento interno de eventos.	RunContext, EventBus, ModuleRegistry, Scheduler.
Adapters	Integrações isoladas por interface; implementação simulada primeiro.	WifiAdapter, BleAdapter, TelemetryAdapter, SiemAdapter, FirewallAdapter.
Storage	Configuração, execuções, achados, aprovações e auditoria.	SQLite repositories, JSON config, append-only audit log.
Reporting	Relatórios reprodutíveis e exportações.	HTML/PDF/JSON/CSV, resumo de KPIs e evidências.

Fluxo operacional obrigatório

- 1. Carregar configuração, verificar integridade e iniciar em modo seguro.
- 2. Validar licença, validade, autorização, allowlist e limites de execução.
- 3. Exibir o escopo efetivo e solicitar confirmação do operador para iniciar.
- 4. Criar RunContext e registrar evento de início antes de chamar qualquer módulo.
- 5. Executar primeiro em simulação; medir telemetria e verificar limiares continuamente.
- 6. Ao exceder limiar, acionar kill-switch, cancelar tarefas e impedir novas ações.
- 7. Produzir relatório, registrar decisões, exceções, resultados e motivo de parada.
- 8. Para mitigação real, exigir aprovação separada e plano de rollback validado.

Estados mínimos da execução

IDLE → VALIDATING → READY → RUNNING → STOPPING → STOPPED; estados adicionais: BLOCKED, FAILED e COMPLETED. Toda transição deve ser explícita, validada e registrada. Uma execução bloqueada não pode continuar por simples repetição de clique.

4. Requisitos não negociáveis de segurança

O documento-base afirma que nenhum módulo ofensivo deve ser executado sem validação prévia de escopo, travas automáticas e trilha de auditoria. filecite turn0file0 L22-L25 Este plano converte essa diretriz em controles testáveis:

- Fail-closed: arquivo scope.json ausente, inválido, expirado ou inconsistente bloqueia a execução.
- Allowlist: cada alvo deve corresponder exatamente a um alvo permitido; não ampliar CIDR nem inferir alvos.
- Modo simulação padrão: módulos novos iniciam em dry-run e não enviam tráfego nem alteram configurações.
- Kill-switch independente: botão visível e atalho de teclado; cancelamento cooperativo e timeout para tarefas.
- Limites conservadores: concorrência, duração, taxa e volume configurados antes de cada execução.
- Limiares: monitorar RTT, erros 5xx, CPU, RAM e perdas; os valores definidos pelo documento-base são configuráveis e precisam de aprovação por ambiente.
- Separação de privilégios: UI e core sem administrador por padrão; ações privilegiadas isoladas em adaptadores.
- Segredos: nunca gravar senhas, tokens ou chaves em logs, JSON de exemplo, código ou relatório.
- Auditoria: registrar usuário local, timestamp UTC, versão, hash de configuração, alvo, módulo, decisão, resultado e motivo de parada.
- Mitigação segura: exigir aprovação humana, visualizar a mudança proposta, validar pré-condições e ter rollback.
- Proteção contra abuso: não incluir funções para roubar credenciais, desautenticar terceiros, interferir em RF, realizar flood contra terceiros ou interceptar tráfego sem autorização.
- Testes em laboratório: testes ativos somente em localhost, VMs/containers isolados ou alvos de teste formalmente autorizados.

Regra de parada inicial

Implementar os gatilhos citados na especificação-base: latência acima de 1.500 ms ou erros HTTP 5xx acima de 5% interrompem a suíte. Acrescentar limites de tempo total, falha de telemetria, cancelamento manual e perda de autorização. Para a primeira versão, se não for possível medir uma métrica essencial, a execução ativa deve parar ou permanecer em simulação.

Nota de governança: um hash encadeado detecta algumas alterações, mas não torna o log inviolável quando o próprio host está comprometido. Para exigência corporativa de imutabilidade, exportar para armazenamento remoto protegido ou SIEM com controles apropriados.

5. Contratos de dados e configuração inicial

O arquivo scope.json deve ser explícito, versionado e validado antes de cada execução. O exemplo abaixo é um modelo de laboratório; os endereços e domínios são ilustrativos e não autorizam testes em sistemas reais.

```
{
  "schema_version": "1.0",
  "engagement_id": "LAB-CHANGE-ME",
  "environment": "local-lab",
  "mode": "simulation",
  "valid_from": "2026-10-29T00:00:00Z",
  "valid_until": "2026-10-30T00:00:00Z",
  "authorized_by": "CHANGE-ME",
  "targets": [
    {
      "type": "host",
      "value": "127.0.0.1",
      "purpose": "local-demo-service"
    }
  ],
  "allowed_modules": ["telemetry_demo", "http_resilience_lab"],
  "limits": {
    "max_duration_seconds": 60,
    "max_concurrency": 2,
    "max_requests_per_second": 2,
    "max_total_requests": 100,
    "max_error_rate_percent": 5,
    "max_rtt_ms": 1500
  },
  "mitigation": {
    "enabled": false,
    "requires_human_approval": true,
    "dry_run": true
  }
}
```

Validações obrigatórias do schema

- Rejeitar propriedades desconhecidas em campos críticos, tipos incorretos, datas inválidas e janela de validade expirada.
- Não permitir curingas globais, alvo vazio, alvo ambíguo ou módulo fora de allowed_modules.
- Normalizar IPs/CIDRs e comparar com biblioteca validada; não usar correspondência textual simples.
- Não permitir alteração de escopo durante uma execução ativa.
- Gerar hash do escopo validado e associá-lo ao RunContext e ao relatório.
- Distinguir ausência de licença, licença inválida e escopo inválido nos códigos de erro.

Eventos mínimos (JSON Lines)

Evento	Campos mínimos
run.started	event_id, timestamp_utc, run_id, operator, module, scope_hash, mode, app_version
scope.validated	run_id, result, policy_version, target_count, allowed_modules, expiration
metric.sample	run_id, timestamp_utc, metric_name, value, unit, source
safety.triggered	run_id, trigger, observed_value, threshold, action_taken
mitigation.proposed	run_id, action_id, target, diff_summary, dry_run, approval_required
run.stopped	run_id, status, reason, elapsed_seconds, counters
finding.created	finding_id, severity, module, evidence_reference, recommendation

6. Interface visual e experiência de uso

A interface deve ser clara para um analista que precisa entender o estado do sistema em poucos segundos. Evitar indicadores decorativos que sugiram segurança sem evidência. Cada KPI deve mostrar fonte, horário da última atualização e estado de confiança (real, importado ou simulado).

Tela	Elementos obrigatórios	Ações
Dashboard	Estado do Core, modo ativo/simulação, última execução, alertas abertos, CPU/RAM e telemetria.	Abrir execução, ver alertas, acessar relatório.
Escopo	Engagement ID, validade, alvos, módulos permitidos, limites, hash e resultado da validação.	Importar, validar, exportar; edição bloqueada durante execução.
Módulos	Nome, estado, versão, capacidades, requisitos e modo suportado.	Abrir detalhes; módulos não implementados aparecem como “Planejado”, não como funcionais.
Nova execução	Escopo validado, módulos escolhidos, limite, dry-run, confirmação explícita.	Pré-visualizar plano; iniciar apenas se as políticas aprovarem.
Console de execução	Estado, tempo decorrido, métricas, eventos, motivo de parada.	Parar; copiar ID da execução; exportar evidências.
Mitigações	Ação proposta, impacto, alvo, diff, aprovação, rollback.	Aprovar/rejeitar; por padrão, só dry-run.
Auditoria	Filtros por data, execução, módulo, gravidade e tipo de evento.	Exportar JSON/CSV; verificar cadeia de hashes.
Relatórios	Resumo, escopo, método, métricas, achados, limitações e recomendações.	Gerar PDF/JSON; sem expor segredos.
Configurações	Caminho de dados, idioma, logs, tema, atualização e diagnóstico.	Salvar com validação e trilha de auditoria.

Requisitos de acessibilidade e robustez

- Layout responsivo ao redimensionamento, navegação por teclado e rótulos acessíveis.
- Cores acompanhadas por texto/ícone; não depender apenas de vermelho/verde.
- Confirmação dupla para parar serviços externos ou aprovar mitigação real.
- Mensagens de erro com causa, código, correção recomendada e ID de correlação.
- Indicador persistente de modo: SIMULAÇÃO, SOMENTE LEITURA ou AÇÃO AUTORIZADA.
- Nenhum botão deve parecer funcional se a capacidade estiver em roadmap.

7. Execução no Windows: desenvolvimento, .bat e pendrive

Para a primeira entrega, manter uma única base de código e oferecer três formas de iniciar. O modo portátil é uma conveniência de distribuição, não uma garantia de segurança: um pendrive pode ser perdido, copiado ou conectado a máquinas não confiáveis.

A. Modo desenvolvimento (VS Code)

- Instalar Python suportado e Git a partir de fontes oficiais; instalar Visual Studio Code e a extensão Python.
- Abrir a pasta raiz no VS Code, criar ambiente virtual `.venv` e instalar dependências fixadas em `requirements.txt` ou `pyproject.toml`.
- Copiar `config.example.json` para `config.json`; nunca guardar segredos no repositório.
- Executar testes automatizados antes de iniciar a aplicação.
- Iniciar pela configuração de depuração do VS Code ou pelo script de desenvolvimento.

B. Inicialização por arquivo .bat

```
@echo off
setlocal
cd /d "%~dp0"
if not exist ".venv\Scripts\python.exe" (
  echo Ambiente virtual nao encontrado.
  echo Execute primeiro scripts\setup_windows.bat
  pause
  exit /b 1
)
".venv\Scripts\python.exe" -m sarr_core
set "RC=%ERRORLEVEL%"
if not "%RC%"=="0" (
  echo A aplicacao terminou com codigo %RC%.
  pause
)
exit /b %RC%
```

C. Script de preparação (setup_windows.bat)

```
@echo off
setlocal
cd /d "%~dp0"
where py >nul 2>nul
if errorlevel 1 (
  echo Python Launcher nao encontrado. Instale uma versao suportada do Python.
  exit /b 1
)
py -3 -m venv .venv
if errorlevel 1 exit /b 1
".venv\Scripts\python.exe" -m pip install --upgrade pip
if errorlevel 1 exit /b 1
".venv\Scripts\python.exe" -m pip install -e .
if errorlevel 1 exit /b 1
".venv\Scripts\python.exe" -m pytest
if errorlevel 1 (
  echo Testes falharam. A instalacao nao sera marcada como validada.
  exit /b 1
)
echo Preparacao concluida e testes aprovados.
pause
```

Os scripts são modelos iniciais. O Copilot deve adaptá-los à estrutura final, não inventar comandos nem presumir que os arquivos já existem. Não executar scripts de instalação com privilégios de administrador sem necessidade comprovada.

D. Modo portátil em pendrive

- Preferência inicial: pacote de aplicação isolado criado a partir do mesmo código, com pasta `data` separada e caminho configurável.
- Evitar incluir tokens, credenciais, certificados privados ou licença secreta no pendrive.
- Adicionar `portable.flag` para que a aplicação use `.\data` ao lado do executável; caso contrário, usar diretório de dados do usuário.

- Na primeira execução, verificar permissões de escrita, espaço livre e versão; não iniciar módulos ativos automaticamente.
- Documentar que o modo portátil pode deixar logs e dados na máquina host caso qualquer adaptador externo os grave fora da pasta portátil.
- Assinar os binários quando houver processo de distribuição formal e gerar hashes SHA-256 dos artefatos.

E. Instalador

Criar instalador apenas depois que o MVP estiver estável. O instalador deve incluir versão, desinstalação, escolha do diretório de dados, criação de atalhos e preservação de relatórios do usuário. Não instalar serviço de sistema ou iniciar automaticamente na inicialização do Windows na primeira versão.

8. Estrutura de pastas sugerida

```
sarr-core/
  pyproject.toml
  README.md
  LICENSE
  .gitignore
  .env.example
  config/
    config.example.json
    scope.example.json
    schemas/
      scope.schema.json
      app-config.schema.json
  src/
    sarr_core/
      __init__.py
      __main__.py
      app.py
      core/
        engine.py
        states.py
        events.py
        run_context.py
        module_registry.py
      safety/
        scope_validator.py
        policy_engine.py
        kill_switch.py
        limits.py
        approvals.py
      domain/
        models.py
        findings.py
        metrics.py
      adapters/
        base.py
        telemetry_simulator.py
        http_lab_adapter.py
        wifi_readonly_adapter.py
        ble_readonly_adapter.py
      services/
        run_service.py
        report_service.py
        audit_service.py
      storage/
        database.py
        repositories.py
        migrations/
      ui/
        main_window.py
        views/
        widgets/
      reporting/
        templates/
  tests/
    unit/
    integration/
    security/
    fixtures/
  scripts/
    setup_windows.bat
    run_windows.bat
    build_portable.bat
  docs/
    architecture.md
    threat-model.md
    operator-guide.md
    release-checklist.md
  artifacts/
    .gitkeep
```

Regras de organização

- Não colocar toda a aplicação em um único arquivo.
- Interfaces dos adaptadores devem ser definidas antes das implementações reais.
- Usar injeção de dependência para trocar adaptadores reais por simuladores nos testes.
- Evitar imports circulares e dependências da UI dentro do núcleo de domínio.
- Manter `artifacts/`, bancos locais e logs fora do Git por padrão.

- Cada módulo deve declarar `implemented`, `simulated`, `read_only` ou `planned`.

9. Fases de desenvolvimento e critérios de aceite

Fase	Entrega	Critérios objetivos de aceite
0 — Preparação	Repositório, ambiente, política de segurança, threat model.	Instalação reproduzível; dependências fixadas; README; nenhum segredo no Git.
1 — Core mínimo	Estados, RunContext, event bus, registro de módulos.	Testes de transição; nenhuma execução sem escopo válido; logs com run_id.
2 — Escopo e kill-switch	Schema, validade, allowlist, limites e cancelamento.	Testes para arquivo ausente, expirado, alvo fora de escopo, timeout e parada manual.
3 — Persistência e auditoria	SQLite, eventos, achados e cadeia de hashes opcional.	Reinício sem corrupção; exportação; logs sem segredos; verificação de integridade.
4 — UI desktop	Dashboard, escopo, módulos, execução, auditoria.	Estados visíveis; navegação básica; erros úteis; modo simulação persistente.
5 — Simuladores	Telemetria fake e servidor HTTP local controlado.	Testes reproduzem RTT alto/5xx e provam parada automática.
6 — Relatórios	PDF/JSON/CSV com evidências e limitações.	Relatório inclui escopo, versão, método, resultados e motivos de parada.
7 — Empacotamento	.bat, modo portátil, pacote de teste.	Executa em máquina limpa de teste; caminho de dados previsível; hash do artefato.
8 — Adaptadores passivos	Integrações Wi-Fi/BLE/telemetria compatíveis e autorizadas.	Permissões documentadas; fallback seguro; teste em hardware autorizado.
9 — Mitigação aprovada	Propostas de firewall/webhook com aprovação e rollback.	Dry-run padrão; simulação de falha; rollback testado; logs completos.
10 — Avaliação de release	Revisão de segurança, desempenho, documentação.	Checklist assinado; riscos residuais conhecidos; versão e changelog.

Não avançar uma fase apenas porque o código compila. A fase só está concluída quando seus critérios de aceite e testes automatizados forem demonstrados.

10. Como usar os prompts no VS Code + Copilot

Use um prompt por vez. Antes de cada fase, abra a pasta do projeto e confirme que o Copilot está trabalhando no repositório correto. Peça que ele leia os arquivos existentes antes de criar ou alterar qualquer coisa. Após a resposta, revise o diff, execute os testes e só então passe para o próximo prompt.

Bloco de instruções que deve acompanhar TODOS os prompts

```
REGRAS OBRIGATÓRIAS PARA ESTA TAREFA:
1. Não alucine. Não invente APIs, bibliotecas, arquivos, comandos, resultados de testes,
   requisitos ou capacidades. Se faltar informação, declare a lacuna e pergunte ou
   proponha uma opção explicitamente marcada como hipótese.
2. Primeiro inspecione os arquivos relevantes do repositório e resuma o que existe.
   Não sobrescreva arquivos sem explicar o motivo e mostrar o impacto.
3. Baseie-se no documento de requisitos e no código existente. Separe fatos,
   decisões propostas e hipóteses. Não declare que algo foi testado sem executar testes.
4. Faça mudanças pequenas e rastreáveis. Antes de editar, apresente o plano e os
   arquivos que pretende criar/alterar. Depois, resuma o diff.
5. Não introduza dependências sem justificar necessidade, licença, versão e alternativa.
   Fixe versões quando apropriado e documente instalação.
6. Segurança por padrão: modo simulação/dry-run, menor privilégio, allowlist,
   limites conservadores, logs sem segredos e falha fechada.
7. Não implemente roubo de credenciais, evasão, persistência furtiva, exploração de
   terceiros, desautenticação, interferência de RF, flood contra sistemas públicos
   ou interceptação não autorizada. Testes ativos ficam restritos a laboratório local
   ou alvos com autorização documentada.
8. Não aplique mudanças reais em firewall, switch, DNS ou dispositivos sem aprovação
   humana explícita, pré-visualização, plano de rollback e testes.
9. Inclua testes unitários para lógica nova e atualize documentação.
10. Ao terminar, informe: arquivos alterados, comandos executados, testes realmente
    executados e seus resultados, limitações conhecidas e próximos passos.
```

Prompt 0 — Análise inicial do repositório

Copiar e colar no Copilot

```
Aplique as REGRAS OBRIGATÓRIAS PARA ESTA TAREFA descritas na conversa.
Estamos iniciando o SARR-Core Engine a partir da especificação técnica fornecida.
Nesta etapa, NÃO implemente funcionalidades ainda. Inspecione o repositório aberto,
identifique arquivos existentes, linguagem, dependências, testes e lacunas. Se o
repositório estiver vazio, diga isso explicitamente. Produza:
1) inventário dos arquivos existentes;
2) riscos e perguntas em aberto;
3) proposta de arquitetura alinhada ao plano;
4) lista mínima de arquivos para a Fase 0;
5) comandos de verificação compatíveis com o ambiente detectado.
Não assuma que módulos Wi-Fi, BLE, SDR, SIEM ou firewall já existem.
Não crie nem execute testes contra alvos externos.
```

Prompt 1 — Preparar projeto e dependências

Copiar e colar no Copilot

```
Aplique as REGRAS OBRIGATÓRIAS PARA ESTA TAREFA. Implemente somente a Fase 0:
estrutura Python, metadados do projeto, .gitignore, README inicial, configuração
de testes e scripts Windows de preparação/execução, conforme o documento de plano.
Antes de editar, liste os arquivos que criará e explique cada um. Escolha versões
de dependências suportadas e verificáveis; se não puder confirmar uma versão, marque
como hipótese e não invente. Não adicione bibliotecas para funções ainda não usadas.
Os scripts devem falhar com mensagens claras, verificar códigos de saída e nunca
solicitar elevação administrativa por padrão. Adicione um teste mínimo de importação.
Execute apenas os testes disponíveis e reporte o resultado real.
```

11. Prompts por fase — Core, segurança e dados

Prompt 2 — Core Engine e máquina de estados

Copiar e colar no Copilot

```
Aplique as REGRAS OBRIGATÓRIAS PARA ESTA TAREFA. Implemente a Fase 1 do SARR-Core:
RunContext, máquina de estados explícita (IDLE, VALIDATING, READY, RUNNING,
STOPPING, STOPPED, BLOCKED, FAILED, COMPLETED), EventBus local e ModuleRegistry.
Defina contratos simples, tipados e documentados. Não inclua chamadas de rede,
shell, rádio ou firewall nesta fase. Cada execução deve ter run_id único, timestamp
UTC, modo de execução e hash do escopo quando disponível. Rejeite transições inválidas.
Escreva testes unitários para transições permitidas/proibidas, isolamento entre
execuções e eventos duplicados. Não alegue cobertura percentual sem medi-la.
```

Prompt 3 — Validação de scope.json e política fail-closed

Copiar e colar no Copilot

```
Aplique as REGRAS OBRIGATÓRIAS PARA ESTA TAREFA. Implemente o schema e o validador
de scope.json com schema_version, engagement_id, environment, mode, janela de
validade, responsável, alvos, módulos permitidos, limites e política de mitigação.
A execução deve ser bloqueada se o arquivo estiver ausente, inválido, expirado,
com alvo fora da allowlist ou módulo não autorizado. Normalize IP/CIDR usando uma
biblioteca conhecida e justifique a dependência. Não amplie escopo automaticamente.
Gere erros estruturados, hash do escopo validado e testes negativos. Use somente
fixtures locais de teste; não examine a rede real do computador.
```

Prompt 4 — Kill-switch, limites e cancelamento

Copiar e colar no Copilot

```
Aplique as REGRAS OBRIGATÓRIAS PARA ESTA TAREFA. Implemente um KillSwitch central
e limites por execução: duração, concorrência, taxa, volume total, RTT e taxa de
erros. Inclua os limiares iniciais do documento-base (RTT > 1500 ms ou HTTP 5xx > 5%)
como configuração, não como constantes espalhadas. Acrescente parada manual,
falha de telemetria, timeout e perda de autorização. Após disparo, bloquear novos
trabalhos, solicitar cancelamento cooperativo e registrar motivo/valores observados.
Não mate processos arbitrários do sistema. Crie testes determinísticos com métricas
simuladas, incluindo limites exatamente iguais ao limiar e acima dele.
```

Prompt 5 — Persistência e trilha de auditoria

Copiar e colar no Copilot

```
Aplique as REGRAS OBRIGATÓRIAS PARA ESTA TAREFA. Implemente persistência local
SQLite para execuções, eventos, métricas, achados e aprovações. Defina migrações
simples, transações e política de retenção configurável. Registre eventos relevantes
em formato estruturado e evite segredos nos logs. Se implementar hash encadeado,
documente que isso não garante inviolabilidade se o host for comprometido. Adicione
exportação JSON Lines e testes de reinício, integridade, dados incompletos e falha
de gravação. Não invente exigências regulatórias nem declare conformidade certificada.
```

Prompt 6 — Modelos de domínio e contratos de adaptadores

Copiar e colar no Copilot

```
Aplique as REGRAS OBRIGATÓRIAS PARA ESTA TAREFA. Defina modelos de domínio para
MetricSample, Finding, RunSummary, ModuleCapability, MitigationProposal e AuditEvent.
Crie interfaces abstratas de adaptadores para telemetria, Wi-Fi somente leitura,
BLE somente leitura, teste HTTP de laboratório, SIEM e firewall. Nesta etapa, implemente
somente simuladores e stubs que informem claramente “não implementado” quando aplicável.
Nunca finja que uma integração externa foi bem-sucedida. Escreva testes de contrato
para cada interface e documente as permissões necessárias para futuras implementações.
```

12. Prompts por fase — interface, simuladores e relatórios

Prompt 7 — Interface gráfica desktop

Copiar e colar no Copilot

```
Aplique as REGRAS OBRIGATÓRIAS PARA ESTA TAREFA. Implemente a interface desktop do
MVP usando a tecnologia já escolhida no projeto; se ainda não houver escolha, compare
brevemente Tkinter e PySide6 e solicite confirmação antes de adicionar dependência.
Crie Dashboard, Escopo, Módulos, Nova Execução, Console, Auditoria, Mitigações,
Relatórios e Configurações. Mostre claramente o modo SIMULAÇÃO, o estado do Core,
a última atualização de telemetria e os módulos como implementado/simulado/planejado.
A UI não deve conter lógica de política de segurança. Conecte-a aos serviços existentes
por interfaces. Implemente botão de parada visível, estados vazios e erros acessíveis.
Não mostre métricas falsas como se fossem reais.
```

Prompt 8 — Simulador de telemetria e teste HTTP local

Copiar e colar no Copilot

```
Aplique as REGRAS OBRIGATÓRIAS PARA ESTA TAREFA. Crie um simulador de métricas e um
servidor HTTP de demonstração restrito a localhost para testar o circuit breaker.
Permita cenários reproduzíveis: RTT normal, RTT acima do limite, taxa 5xx acima do
limite, telemetria ausente e cancelamento manual. Limite duração, concorrência,
taxa e total de requisições. Não implemente SYN/UDP flood, Slowloris, esgotamento de
conexão, tráfego distribuído nem execução contra host remoto. Inclua fixtures e testes
que provem a interrupção. A UI deve identificar os dados como simulados.
```

Prompt 9 — Relatórios de auditoria

Copiar e colar no Copilot

```
Aplique as REGRAS OBRIGATÓRIAS PARA ESTA TAREFA. Implemente relatório de execução
com: identificação e versão do sistema, operador, hash e validade do escopo, módulos,
modo, horário de início/fim, métricas, achados, evidências referenciadas, ações de
segurança disparadas, motivo de parada, limitações e recomendações. Adicione exportação
JSON e CSV; PDF somente se a dependência e o layout forem justificados. Escapar conteúdo
importado para evitar injeção no relatório. Remover tokens, senhas, chaves e dados
sensíveis. Não escrever “aprovado” ou “seguro” sem critérios de aceite explícitos.
Crie testes de campos obrigatórios, dados vazios e conteúdo malformado.
```

Prompt 10 — Módulos Wi-Fi e BLE somente leitura

Copiar e colar no Copilot

```
Aplique as REGRAS OBRIGATÓRIAS PARA ESTA TAREFA. Avalie como implementar adaptadores
passivos/readonly para inventário Wi-Fi e BLE na plataforma e no sistema operacional
identificado. Antes de codificar, informe as limitações de hardware, drivers, permissões,
APIs e sistema operacional; não presuma que a máquina possui adaptador compatível.
Nesta fase, priorize importação de arquivos de inventário e simuladores. Se uma API real
não puder ser verificada, deixe um stub documentado. Não capture credenciais, não execute
desautenticação, não tente pareamento não autorizado e não interaja com dispositivos
fora da autorização. Escreva testes usando dados sintéticos.
```

Prompt 11 — Detecção de anomalias explicável

Copiar e colar no Copilot

```
Aplique as REGRAS OBRIGATÓRIAS PARA ESTA TAREFA. Crie um primeiro detector de anomalias
para séries temporais usando métodos estatísticos simples e explicáveis. Não adicione
modelos de IA pesados sem demonstrar necessidade e dependências. Defina janela,
baseline, limiar, tratamento de valores ausentes e explicação do alerta. No MVP, o motor
apenas recomenda ações; não altera firewall nem rate limiting. Teste com dados sintéticos
normais, picos, sazonalidade, dados faltantes e falsos positivos. Documente limitações
e não alegue precisão sem avaliação quantitativa.
```

13. Prompts por fase — mitigação, empacotamento e qualidade

Prompt 12 — Mitigação com aprovação humana e rollback

Copiar e colar no Copilot

```
Aplique as REGRAS OBRIGATÓRIAS PARA ESTA TAREFA. Implemente somente o fluxo de proposta
de mitigação: gerar diff legível, explicar impacto, mostrar alvo, exigir aprovação humana,
verificar escopo novamente, registrar aprovador e oferecer rollback documentado.
O comportamento padrão é dry-run e nenhum comando real deve ser executado. Crie
adaptadores simulados para firewall, WAF e switch. Qualquer execução real futura deve
ficar atrás de uma interface separada, configuração explícita, credenciais seguras e
testes aprovados. Não use subprocess com strings montadas; se um adaptador privilegiado
vier a ser aprovado, exigir argumentos estruturados, allowlist de comandos e validação.
Teste rejeição, aprovação, timeout, rollback e falha parcial.
```

Prompt 13 — SIEM e webhooks

Copiar e colar no Copilot

```
Aplique as REGRAS OBRIGATÓRIAS PARA ESTA TAREFA. Defina uma interface de saída de eventos
para SIEM e webhook, com implementação simulada local. Antes de qualquer conector real,
peça a documentação oficial da API, esquema de autenticação, ambiente e autorização.
Não invente endpoints, payloads ou nomes de campos. Inclua timeout, retry limitado,
backoff, idempotência, proteção contra duplicidade e redaction de segredos. Não permitir
que eventos recebidos acionem comandos arbitrários. Teste indisponibilidade, respostas
inválidas e vazamento de dados.
```

Prompt 14 — Scripts .bat e modo portátil

Copiar e colar no Copilot

```
Aplique as REGRAS OBRIGATÓRIAS PARA ESTA TAREFA. Revise a estrutura real do projeto e
implemente scripts Windows consistentes com ela: setup_windows.bat, run_windows.bat
e build_portable.bat, se o empacotamento escolhido permitir. Não presuma caminhos,
executáveis ou módulos que não existam. Os scripts devem usar caminhos relativos ao
próprio arquivo, verificar pré-requisitos, preservar códigos de saída, exibir erros
claros e não executar como administrador. Para modo portátil, suporte portable.flag,
pasta data ao lado do app, teste de escrita e fallback documentado. Não incluir segredos
ou dados reais no pacote. Adicione instruções de build, verificação de hash e teste em
máquina limpa; não declare o pacote portátil antes de validá-lo.
```

Prompt 15 — Testes de segurança e revisão

Copiar e colar no Copilot

```
Aplique as REGRAS OBRIGATÓRIAS PARA ESTA TAREFA. Faça uma revisão de segurança do código
existente sem ampliar o escopo. Procure falhas de autorização, bypass de scope.json,
condições de corrida no kill-switch, caminhos inseguros, logs com segredos, subprocess
perigoso, dependências desnecessárias, validação insuficiente e ações reais ativadas por
padrão. Crie ou ajuste testes de regressão para cada problema confirmado. Não afirme que
o sistema está livre de vulnerabilidades. Classifique achados por severidade, evidência,
impacto, pré-condição e correção. Separe problemas confirmados de hipóteses.
```

Prompt 16 — Preparação de release

Copiar e colar no Copilot

```
Aplique as REGRAS OBRIGATÓRIAS PARA ESTA TAREFA. Prepare checklist de release para o
SARR-Core MVP. Verifique documentação, dependências fixadas, testes, logs, configuração
de exemplo, scripts Windows, build portátil, licença de dependências, changelog,
versionamento e política de atualização. Execute apenas verificações disponíveis no
ambiente e registre os resultados reais. Gere uma lista de bloqueadores e itens
pendentes. Não assine, publique, distribua ou declare conformidade sem autorização.
A release deve iniciar em modo simulação e documentar limitações conhecidas.
```

14. Casos de uso do documento-base convertidos em testes

O documento-base descreve quatro casos de uso principais: auditoria Wi-Fi e rogue AP; mapeamento de atributos BLE IoT; teste de carga L7 com circuit breaker; e mitigação ativa/isolamento LAN. filecite turn0file0 L118-L137 Para o MVP, validar a lógica com fixtures e simuladores antes de conectar a hardware ou infraestrutura real.

Caso	Teste MVP	Resultado esperado
UC-01 Wi-Fi / Rogue AP	Carregar inventário sintético com BSSID autorizado e um BSSID desconhecido.	Classificar discrepância, registrar evidência e emitir alerta simulado; não transmitir pacotes.
UC-02 BLE / GATT	Carregar inventário sintético de serviços e atributos.	Marcar serviços críticos configurados como abertos conforme política de teste; não conectar a dispositivo real.
UC-03 HTTP L7 / Circuit Breaker	Servidor localhost com cenários de RTT alto e 5xx.	Parar automaticamente ao cruzar o limiar configurado e registrar o motivo.
UC-04 ARP / Mitigação LAN	Injetar eventos ARP/MAC sintéticos no simulador.	Gerar proposta de mitigação em dry-run, exigir aprovação e apresentar rollback.

Matriz de critérios de aceitação

Critério	Condição de aprovação	Evidência
Escopo	Nenhum módulo inicia com escopo ausente, expirado ou inválido.	Testes negativos automatizados.
Kill-switch	A execução para no limiar, por botão e em falha de telemetria.	Logs de teste e relatório.
Limites	Taxa, concorrência, duração e volume nunca ultrapassam configuração.	Contadores e asserts.
UI	Modo e estado são sempre visíveis; módulo planejado não parece operacional.	Teste manual documentado e screenshots de QA.
Auditoria	Toda execução tem início, fim e motivo; eventos são correlacionáveis.	Exportação JSONL verificada.
Mitigação	Dry-run padrão; aprovação e rollback disponíveis antes de execução real.	Testes com adaptador simulado.
Distribuição	Scripts falham com clareza e pacote roda em ambiente limpo de teste.	Log do teste de instalação.
Relatório	Escopo, método, limitações, métricas e achados estão presentes.	PDF/JSON de exemplo sem dados sensíveis.

15. Operação, riscos e decisões ainda necessárias

O documento-base não define todos os detalhes necessários para uma implementação completa. Os pontos abaixo precisam de decisão do responsável técnico antes de integrar módulos ativos.

Questão	Por que precisa ser definida	Padrão seguro provisório
Sistema operacional-alvo	Acesso a Wi-Fi/BLE e empacotamento variam por OS.	Windows para UI e MVP; adaptadores por capacidade.
Licença e assinatura	O documento exige arquivo de licença, mas não especifica formato, emissor ou verificação.	Bloquear execução ativa até definir modelo; em desenvolvimento usar licença de laboratório explícita.
Formato de autorização	É preciso comprovar quem autorizou, alvos, janela e módulos.	scope.json validado + registro de change/engagement.
SIEM/WAF/switch	APIs, autenticação e rollback dependem do fornecedor.	Stubs e simuladores; não inventar endpoints.
Hardware SDR/Wi-Fi/BLE	Compatibilidade depende de adaptador, driver, região e sistema.	Importação de dados/simulação até inventário de hardware.
Requisitos legais e regulatórios	RF e testes de carga podem afetar terceiros ou serviço crítico.	Revisão responsável antes de qualquer teste ativo.
Modelo de IA	Não há dataset, baseline ou métrica de avaliação definidos.	Estatística explicável e recomendação sem ação automática.
Assinatura de release	Necessária para distribuição confiável em contexto corporativo.	Hash SHA-256 e processo de assinatura posterior.

Riscos principais

- Falso positivo: AP, dispositivo ou comportamento legítimo pode ser classificado como suspeito. Mitigação: evidências, nível de confiança e revisão humana.
- Indisponibilidade: teste de carga pode degradar serviço. Mitigação: localhost/lab, limites conservadores e circuit breaker testado.
- Bloqueio indevido: resposta automática pode interromper operação legítima. Mitigação: dry-run, aprovação, janela de mudança e rollback.
- Falsa sensação de imutabilidade: logs locais podem ser alterados por administrador/atacante. Mitigação: exportação remota protegida.
- Portabilidade insegura: pendrive pode conter dados sensíveis. Mitigação: criptografia do volume, minimização e não armazenar segredos.

16. Checklist de início e definição de pronto

Checklist antes de iniciar

- Criar repositório privado e confirmar que não há segredos ou dados de produção.
- Adicionar este PDF e o documento-base à pasta docs/ para consulta controlada.
- Configurar Python, Git, VS Code, extensão Python e Copilot.
- Usar o Prompt 0 para inventariar o repositório; não pedir ao Copilot para gerar tudo de uma vez.
- Criar Fase 0, executar testes e revisar o diff antes de continuar.
- Definir responsável, ambiente de laboratório e processo de autorização.
- Manter todos os módulos ativos desabilitados até os controles de escopo e kill-switch passarem nos testes.

Definição de pronto para o MVP

- A aplicação abre por interface gráfica e também pode ser iniciada pelo .bat.
- A tela deixa explícito que módulos de rede/radio reais podem estar apenas planejados ou simulados.
- scope.json é validado; escopo inválido bloqueia a execução.
- Execução simulada gera eventos, métricas, achados e relatório exportável.
- O kill-switch funciona por limiar e por ação manual.
- Testes unitários e de integração passam no ambiente de release, com resultados registrados.
- Pacote portátil é testado em máquina limpa e não contém credenciais.
- Documentação explica instalação, execução, parada, backup, limitações e recuperação.

Ordem prática recomendada

Semana/etapa 1: Prompt 0 → Prompt 1 → revisão. Etapa 2: Prompts 2–6 e testes. Etapa 3: Prompts 7–9 para interface e demonstração funcional. Etapa 4: Prompt 14 para .bat/portabilidade, após a estrutura estar estável. Etapa 5: revisão de segurança e release. Só depois planejar integrações reais e mitigação.

Referência de origem

Documento fornecido: “Documento de Arquitetura, Casos de Uso e Fluxogramas.pdf”, contendo a especificação “SARR-Core Engine — Plataforma de Resiliência Defensiva & Security Testing”, versão 1.2.0-FINAL. Os requisitos de arquitetura, vetores, casos de uso, limiares e roadmap foram organizados a partir desse material; decisões de tecnologia e fases de implementação neste plano são recomendações de engenharia, não trechos literais do documento-base.

Fim do documento — Plano de implementação SARR-Core Engine, versão 1.0.